

Ahead of the Curve

Identifying Frontier Models Before the Crowd Does

Noah Zemljic

1 Problem Statement

Hugging Face has become the de facto platform for open-source AI models, with hundreds of new models published every day. Deciding which models are worth the cost of evaluation, fine-tuning, or deployment before significant credibility has been projected creates a real operational challenge for companies and research institutions. A wrong bet not only costs time and compute, but it forces engineering teams to constantly scout for replacements and actively undermines the productivity gains they were hoping these models would deliver.

This project builds a live ML system that makes that decision automatically. Given a model uploaded in the last 24 hours, the system predicts both its expected download count and whether the model will land in the top quartile of download growth within 30 days. The range of topics is deliberately narrow, with focus lying on three frontier niches where the cost of a wrong call is greatest: **general-purpose robotics** (VLA models and embodied agents), **small language models and edge inference** as well as **multimodal reasoning**.

2 Data Source

All data is fetched in real time from the official Hugging Face Hub API via the `huggingface_hub` Python library. The API exposes everything this project needs: model identifiers, creation and modification timestamps, rolling 30-day and cumulative download counts, likes, Hugging Face's own trending score, tags, library name, and full model card text.

An Airflow workflow will run at 00:00 UTC each day and fetch all frontier-matching models modified in the last 24 hours. From these models, features will be computed, and a new snapshot will be appended to the feature store. On the first run, the pipeline back-fills 90 days of history to give the training pipeline enough data to learn from immediately, without having to wait a month for live labels to accumulate.

3 Features

For each tracked model, two categories of features are computed and stored as a timestamped snapshot:

- **Semantic relevance** — The model card text is embedded using a sentence transformer and scored against frontier topic vectors via cosine similarity. Models scoring below 0.25 are discarded before entering the feature store.
- **Momentum** — The model's age in hours since creation, and its early download velocity in the first 24 and 72 hour windows.

Once a snapshot is 30 days old, two labels are appended: the raw download growth between day 0 and day 30, and a binary flag indicating whether that growth lands in the top quartile of the same-day cohort. These are the regression and classification targets respectively.

Prediction requires no waiting. On first run, the pipeline back-fills 90 days of history, providing enough labels to train immediately. From that point, any new model that clears the relevance gate receives a prediction (expected 30-day downloads and a high/low adoption signal) at the moment of upload.

4 Tech Stack

The tech stack for this project is as follows:

- **Object Store / Feature Store:** Hopsworks
- **Model Registry & Experiment Tracking:** MLFlow
- **Orchestration:** Apache Airflow
- **Containerization:** Docker
- **Versioning:** GitHub / DagsHub

5 Architecture

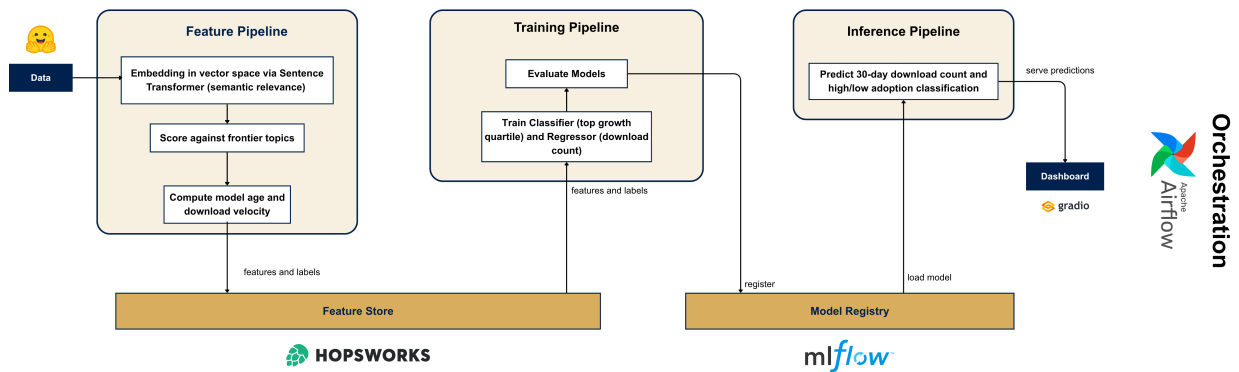


Figure 1: FTI pipeline architecture

6 GitHub Repository

The GitHub Repository for this project is available [here](#).